App Dev - 2 Project Report

1. Student Details

Name: Parth Jain

Roll Number: 23f2003877

Email: 23f2003877@ds.study.iitm.ac.in

About Me: I am a student at IIT Madras BS Degree program with a strong interest in full-stack web development, machine learning, and building production-grade applications. I enjoy creating scalable systems that solve real-world problems using modern technologies.

---

2. Project Details

Project Title: Placement Portal Application (PPA V2)

Problem Statement: To design and build a multi-role web application that streamlines the campus placement process by connecting students, companies, and administrators. The platform enables companies to post placement drives, students to apply and track their applications, and admins to manage approvals, generate reports, and monitor the entire placement lifecycle.

Approach: The app was built using Flask as the backend REST API framework with a modular blueprint structure. Vue.js 3 (CDN) powers the reactive single-page frontend. Redis provides API response caching, while Celery handles asynchronous background tasks such as CSV export, monthly report generation, and daily email reminders. JWT-based authentication secures all API endpoints with role-based access control.

---

3. AI/LLM Declaration

I used **Chatgpt** to assist in debugging edge cases, generating css styling and debugging html structure, and improving my database design. The extent of AI/LLM usage is around **15–20%**, limited to **code suggestions, debugging assistance, and css styling** . All core architecture decisions, integration logic, database design, and testing were done manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework for REST APIs
SQLAlchemy	Object Relational Mapper for SQLite database
SQLite	Lightweight relational database for data storage
Vue.js 3 (CDN)	Reactive frontend framework for building SPA
Bootstrap 5	Frontend styling, responsive design, and UI components
Chart.js	Data visualization (bar/pie charts) on admin dashboard
Redis	In-memory caching for API responses (TTL-based)
Celery	Distributed task queue for background job processing
Flask-Mail	Email delivery for reports and notifications
PyJWT	JSON Web Token authentication (stateless)
Flask-CORS	Cross-Origin Resource Sharing for API access
xhtml2pdf	HTML to PDF conversion for monthly reports
gradio_client	Integration with HuggingFace Space for ATS resume check
PyPDF2	PDF text extraction from uploaded resumes
Werkzeug	Password hashing (PBKDF2-SHA256) and file security

---

5. Database Schema / ER Diagram

Tables:

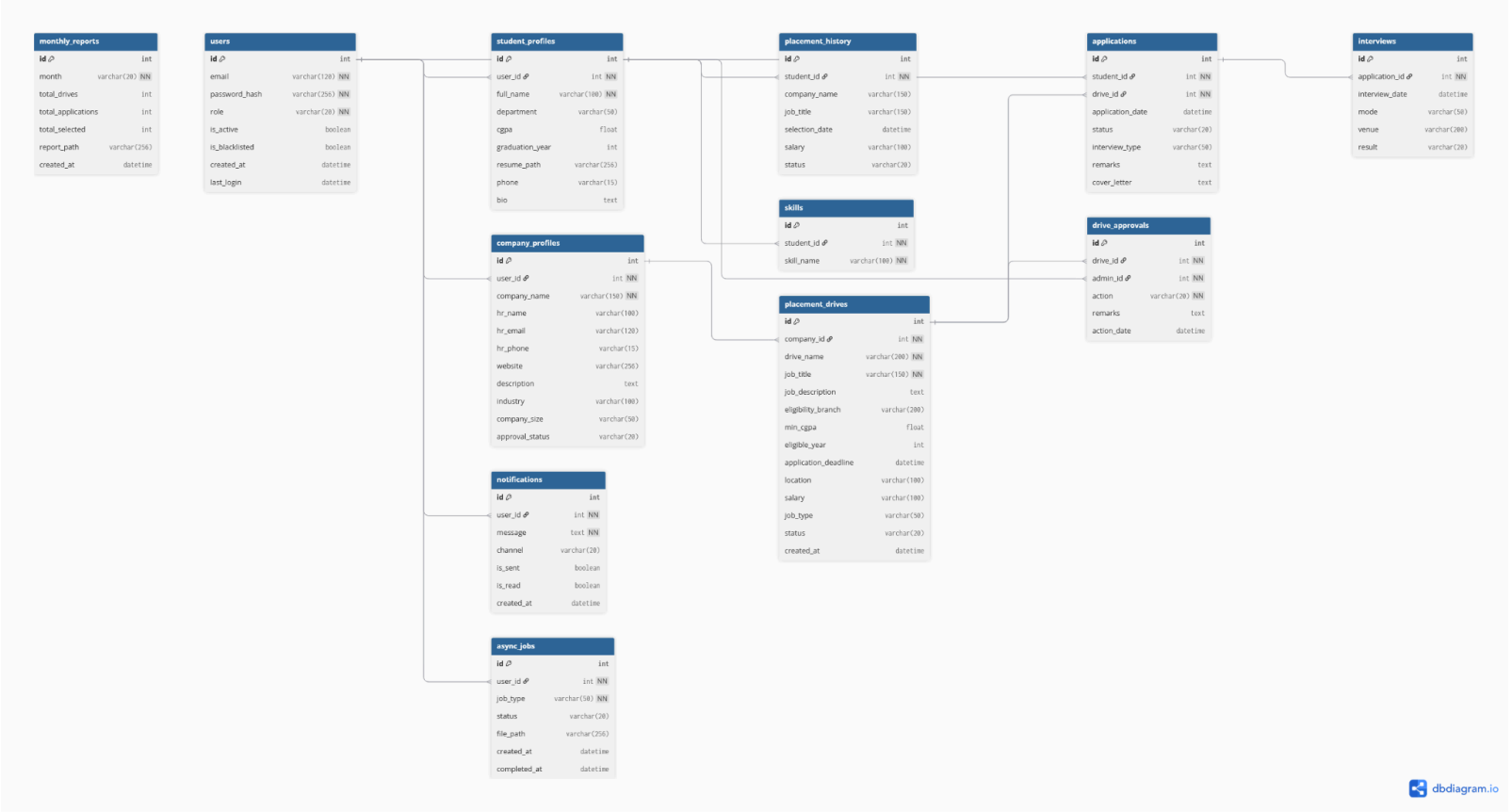
Table	Description	Key Columns
User	Stores authentication and role ir	id, email, password_hash, role (admin/student/company), is_active is_blacklisted
StudentProfile	Student details linked to User	id, user_id (FK), full_name, department, cgpa, resume_path, graduation_year
CompanyProfile	Company details linked to User	id, user_id (FK), company_name, hr_name, website, industry, approval_status
PlacementDrive	Job postings by companies	id, company_id (FK), drive_name, job_title, job_type, min_cgpa, status, application_deadline
Application	Student applications to drives	id, student_id (FK), drive_id (FK), status (pending/shortlisted/selected/reject
Interview	Scheduled interviews	id, application_id (FK), interview_date, mode, venue, result
DriveApproval	Admin approval/rejection log	id, drive_id (FK), admin_id (FK), action, remark
MonthlyReport	Generated placement reports	id, month, total_drives, total_applications, total_selected, report_path
Notification	In-app and email notifications	id, user_id (FK), message, channel, is_read
AsyncJob	Background job tracking	id, user_id (FK), job_type, status, file_path
Skill	Student skills	id, student_id (FK), name
PlacementHistory	Past placement records	id, student_id (FK), company_name, job_title, s

Relationships:

- One-to-One → **User** → **StudentProfile** / **User** → **CompanyProfile**
- One-to-Many → **User** → **Notifications**
- One-to-Many → **CompanyProfile** → **PlacementDrives**
- One-to-Many → **PlacementDrive** → **Applications**
- One-to-Many → **Application** → **Interviews**
- One-to-Many → **StudentProfile** → **Applications**
- Unique Constraint → **(student_id, drive_id)** prevents duplicate applications

Design Rationale:

The schema follows a normalized relational design with foreign key constraints and cascade deletes to maintain referential integrity. The User table uses a single-table inheritance pattern with a role column to distinguish between admin, student, and company users, simplifying authentication while keeping role-specific data in separate profile tables.



(taken form dbdiagram.io)

---

6. API Resource Endpoints

The application exposes **40+ RESTful API endpoints** organized across three Flask Blueprints:

Authentication (app.py):

Endpoint	Method	Description

/api/auth/login	POST	Authenticate user and generate JWT token
/api/auth/logout	POST	Invalidate user session
/api/auth/me	GET	Verify token and return current user info

Admin Routes (routes/admin.py):

Endpoint	Method	Description
/api/admin/dashboard	GET	Aggregated stats (cached in Redis)
/api/admin/companies/pending	GET	List companies awaiting approval
/api/admin/companies/{id}/approve	PUT	Approve a company registration
/api/admin/drives/pending	GET	List drives awaiting approval
/api/admin/drives/{id}/approve	PUT	Approve a placement drive
/api/admin/reports/generate	POST	Trigger async monthly report generation
/api/admin/reports/download/{id}	GET	Download generated report PDF

Student Routes (routes/student.py):

Endpoint	Method	Description

/api/student/register	POST	Student registration with validation
/api/student/drives	GET	Browse approved drives (with search/filter)
/api/student/drives/{id}/apply	POST	Apply for a placement drive
/api/student/export	POST	Export applications as CSV (async Celery task
/api/student/ats-check	POST	AI-powered resume match analysis

Company Routes (routes/company.py):

Endpoint	Method	Description
/api/company/register	POST	Company registration
/api/company/drives	POST	Create a new placement drive
/api/company/applications/{id}/status	PUT	Shortlist/select/reject applications
/api/company/applications/{id}/interview	POST	Schedule interview for applicant

---

7. Architecture and Features (optional)

Architecture Overview:

```
backend/
├── app.py              - Flask application factory, auth routes, entry point
```

```
├── config.py           - Configuration (DB, JWT, Redis, Celery, Mail)
├── models.py           - 12 SQLAlchemy models with relationships
├── auth.py             - JWT token generation, validation, decorators
├── cache.py            - Redis caching (singleton, TTL-based)
├── validators.py       - Input validation (email, password, phone, CGPA)
├── celery_worker.py    - Celery configuration with beat schedule
├── tasks.py            - 3 background tasks (CSV export, report, reminders)
├── routes/
│   ├── admin.py        - Admin management endpoints (Blueprint)
│   ├── student.py      - Student-facing endpoints (Blueprint)
│   └── company.py      - Company-facing endpoints (Blueprint)
├── instance/ppa.db     - SQLite database file
├── exports/            - Generated CSV export files
└── reports/            - Generated HTML/PDF report files


frontend/
├── templates/
│   └── index.html      - Single Jinja2 template (Vue.js SPA entry point)
└── static/
    ├── js/app.js       - Vue 3 application (676 lines, 50+ methods)
    └── css/style.css   - Custom design system (765 lines, CSS variables)
```

The project follows a **separation of concerns** pattern: Flask serves only JSON APIs (no server-side rendering), while Vue.js handles all UI rendering on the client side. The single `index.html` Jinja2 template is used only as the entry point to load CDN libraries and the Vue app.

Implemented Features:

Default Features:

- Multi-role authentication (Admin, Student, Company) with JWT
- Company registration with admin approval workflow
- Placement drive creation with admin approval
- Student application to drives with eligibility checks
- Application status management (shortlist, select, reject)
- Interview scheduling with date, mode, and venue
- In-app notification system
- Input validation on both client and server side
- Responsive UI with dark mode support

Additional Features:

- **Redis Caching:** API responses cached with TTL for performance optimization
- **Celery Background Jobs:** CSV export, monthly report generation, daily email reminders
- **Monthly PDF Reports:** Auto-generated with statistics, saved to filesystem, emailed to admin, and sent via Google Chat webhook
- **ATS Resume Check:** AI-powered resume-job description matching using a HuggingFace Space with sentence-transformer embeddings
- **Async Job Polling:** Frontend polls for background task completion and auto-downloads results
- **Chart.js Dashboard:** Visual analytics with bar and pie charts on admin dashboard
- **Search and Filtering:** Full-text search across drives with type/status filters

---

8. Video Presentation

Drive Link:

MAD 2 PPA.mp4
